🔌 Fonctionnement Détaillé de la Communication : n8n ↔ App Eco-Park

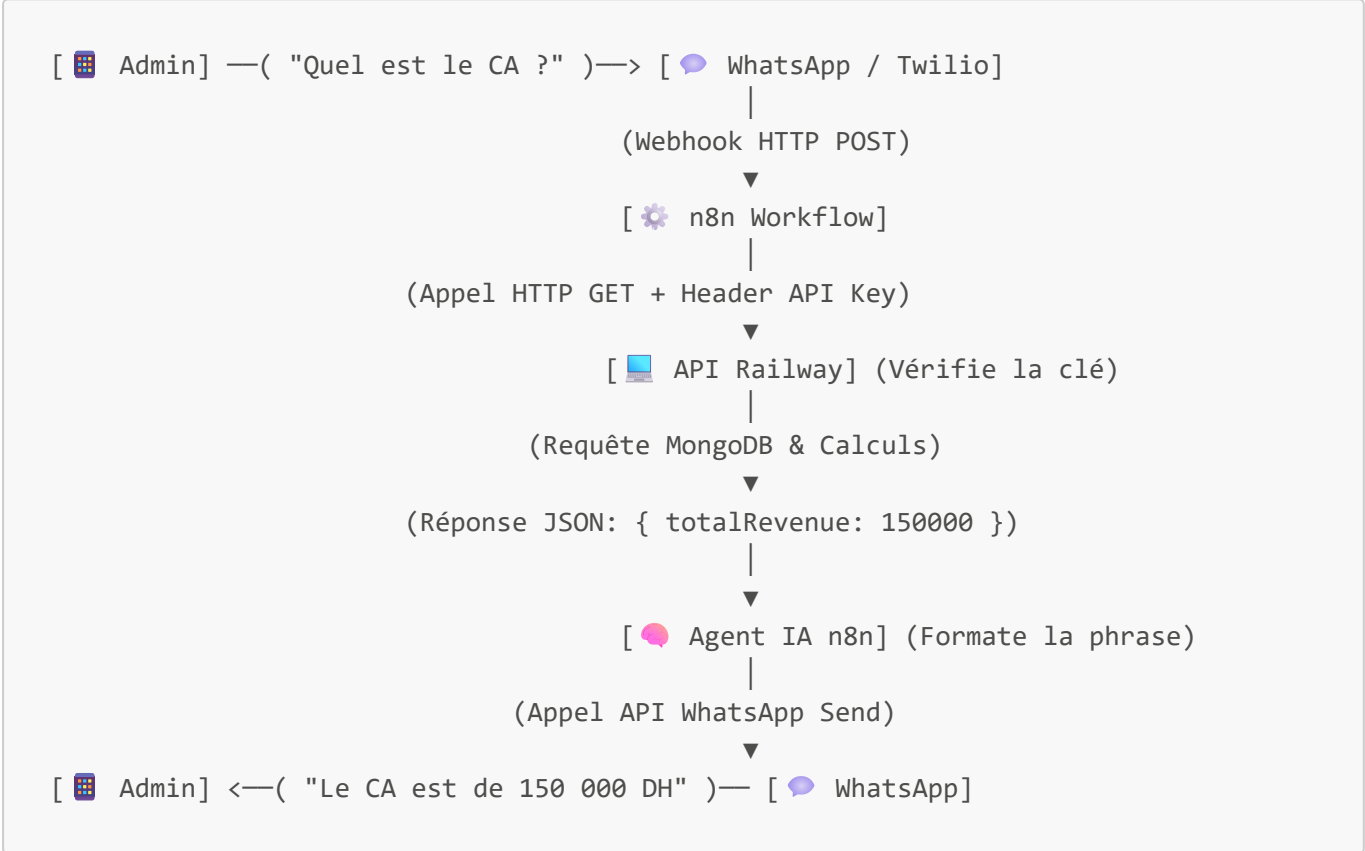
La communication entre **n8n** et ton application se fait via des requêtes HTTP REST sécurisées. n8n agit comme un "client" (comme le ferait ton frontend React ou Postman) et appelle les routes de ton backend Express sur Railway.

Voici le fonctionnement technique détaillé, de la sécurité à la transmission de données.

---

1. Le Flux de Communication (Séquence)

Quand l'admin pose une question sur WhatsApp, voici le parcours exact de la donnée :



---

2. La Sécurité : Comment authentifier n8n ?

Actuellement, ton application utilise des tokens JWT (`Authorization: Bearer ...`) associés à des utilisateurs physiques (gérants, admins) qui expirent au bout de quelques heures/jours.

Pour n8n, on utilise un **Token de Service permanent (API Key)**.

Le Code à ajouter dans ton Backend (`backend/middleware/auth.js`)

On crée un middleware simple qui autorise soit un token JWT (gérant sur l'app), soit la clé d'API n8n.

```
// backend/middleware/auth.js (exemple de logique d'authentification double)

export const protectOrApiKey = async (req, res, next) => {
  // 1. Vérification de la Clé d'API n8n
  const apiKey = req.headers['x-api-key'];
  if (apiKey && apiKey === process.env.N8N_API_KEY) {
    req.user = { role: 'admin', fullName: 'Système n8n' }; // On simule un
utilisateur admin
    return next();
  }

  // 2. Fallback sur le JWT classique (si pas de clé d'API)
  let token;
  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer'))
{
    try {
      token = req.headers.authorization.split(' ')[1];
      const decoded = jwt.verify(token, process.env.JWT_SECRET);
      req.user = await User.findById(decoded.id).select('-password');
      return next();
    } catch (error) {
      return res.status(401).json({ message: 'Non autorisé, token invalide' });
    }
  }

  return res.status(401).json({ message: 'Non autorisé, clé ou token manquant' });
};
```

Dans Railway, il te suffira d'ajouter une variable d'environnement `N8N_API_KEY=ma_cle_super_secrete_123`.

---

3. Configuration du Node dans n8n

Dans n8n, tu utiliseras le node **"HTTP Request"** comme un outil (Tool) pour l'Agent IA.

Paramètres du Node n8n :

- **Method** : `GET`
- **URL** : `https://eco-park-production.up.railway.app/api/dashboard/kpis`
- **Headers** (Entêtes) :
  - `Content-Type` : `application/json`
  - `x-api-key` : `ma_cle_super_secrete_123` *(récupérée depuis les variables d'env n8n)*
- **Query Parameters** :
  - `month` : `{{$parameter.month}}` *(extrait dynamiquement par l'IA depuis la question de l'admin)*

---

4. Format d'Échange des Données (Exemple)

La Requête envoyée par n8n :

```
GET /api/dashboard/kpis?month=2026-06 HTTP/1.1
Host: eco-park-production.up.railway.app
x-api-key: ma_cle_super_secrete_123
Content-Type: application/json
```

La Réponse renvoyée par ton Backend :

Ton backend renvoie exactement le JSON que nous avons inspecté dans `dashboard.js` :

```
{
  "month": "2026-06",
  "totalRevenue": 150000,
  "totalPurchases": 45000,
  "totalSalaries": 35000,
  "totalExpenses": 15000,
  "netProfit": 55000,
  "stockAlertCount": 1,
  "stockAlerts": [
    { "productName": "FARINE (Crêperie)", "remainingStock": 5 }
  ]
}
```

L'interprétation par l'Agent IA :

L'Agent IA dans n8n reçoit ce JSON brut. Grâce aux instructions système (Prompt) que tu lui donnes, il convertit ce JSON en texte naturel :

> *"Le chiffre d'affaires est de 150 000 DH avec un bénéfice de 55 000 DH. Attention, il reste 5 kg de farine à la crêperie."*

Summary : Pourquoi c'est facile et puissant ?

- **Pas de duplication de code** : Tout le calcul (les sommes MongoDB, les jointures) reste dans Express.
- **Temps réel** : Dès qu'une serveuse ou un gérant modifie un shift ou saisit une facture sur l'application Web, l'information est instantanément disponible dans WhatsApp.
- **Sécurisé** : Seul ton serveur n8n (qui possède la clé secrète) et ton numéro de téléphone WhatsApp peuvent interroger l'API.